RAG-Based Customer Support Assistant

---

1. Introduction

This document describes the internal design and implementation details of the system components.

---

2. Module Design

2.1 Document Processing Module

- Loads and parses PDF documents
- Outputs raw text

2.2 Chunking Module

- Splits text into smaller segments
- Adds overlap for context preservation

2.3 Embedding Module

- Converts text chunks into vectors
- Uses HuggingFace models

2.4 Vector Storage Module

- Stores embeddings in ChromaDB
- Supports similarity search

2.5 Retrieval Module

- Hybrid retrieval:
    - Semantic search
    - Keyword search

2.6 Query Processing Module

- Accepts user input
- Maintains session memory

2.7 Graph Execution Module

- Executes LangGraph workflow
- Handles state transitions

2.8 HITL Module

- Triggers human intervention
- Integrates manual responses

---

3. Data Structures

3.1 Document Structure

```
{
  "content": "text",
  "metadata": {}
}
```

3.2 Chunk Structure

```
{
  "text": "chunk content",
  "embedding": []
}
```

3.3 Graph State

```
{
  "query": str,
  "intent": str,
  "context": list,
  "response": str,
  "confidence": float,
  "escalate": bool,
  "history": list
}
```

---

4. Workflow Design

4.1 Nodes

- Retrieve

- Intent Classification
- Billing Agent
- Support Agent
- Evaluate
- HITL
- Memory

4.2 Edges

- Retrieve → Intent
- Intent → Agent
- Agent → Evaluate
- Evaluate → HITL or Memory

4.3 State Management

State object is passed and updated across nodes.

---

5. Conditional Routing Logic

5.1 Agent Routing

- Billing-related queries → Billing Agent
- Other queries → Support Agent

5.2 Escalation Logic

- Confidence < 0.6
- No relevant context
- Complex queries

---

6. API Design

6.1 Input

```
{
  "query": "string",
  "session_id": "string"
}
```

6.2 Output

```
{
  "answer": "string",
  "confidence": 0.8,
  "escalated": false
}
```

---

7. Error Handling

- Missing context → escalate
- Invalid input → validation error
- LLM failure → fallback model